RUNGTA INTERNATIONAL SKILLS UNIVERSITY
School of Information Technology
BACHELOR OF COMPUTER APPLICATION (BCA)

Project Practical Submission Report

PS NO: 98

Student Name	Shreya Singh	**ERP Number**	RU-26-04631
Academic Session	2026-27	**Section**	Section B
Submitted To	Satyaprakash Sir	**Project Title**	**Number Guessing Game**

PROJECT SPECIFICATIONS & REQUIREMENTS

Problem Statement	Create a game in which the computer picks a random number between 1 and 100 (fixed to 42 for testing) and the player guesses it within 5 attempts, receiving 'Too high' or 'Too low' hints.
Language & Concepts	**Language:** Python 3 | **Concepts:** Loops, Conditions, Functions, Validation.

SOURCE CODE (ONE.PY)

```
import random

def play_game():
    secret_number = 42  # Fixed secret number for testing requirement
    max_attempts = 5
    attempts_left = max_attempts
    attempt_count = 0

    while attempts_left > 0:
        user_input = input("Enter your guess: ").strip()

        # Step 1: Validate numeric input
        if not user_input.lstrip('-').isdigit():
            print("Invalid input. Enter a whole number.")
            continue

        guess = int(user_input)

        # Step 2: Validate range (1 to 100)
        if guess < 1 or guess > 100:
            print("Out of range. Enter a number between 1 and 100.")
            print(f"Attempts left: {attempts_left} (no attempt lost)")
            continue

        attempt_count += 1
        attempts_left -= 1

        # Step 3: Check guess outcome
        if guess == secret_number:
            print(f"Congratulations! You guessed it in {attempt_count} attempt{'s' if attempt_
            break
        elif guess < secret_number:
            if attempts_left > 0:
                print(f"Too low! Attempts left: {attempts_left}")
            else:
                print("Too low")
        else:
            if attempts_left > 0:
                print(f"Too high! Attempts left: {attempts_left}")
            else:
                print("Too high")

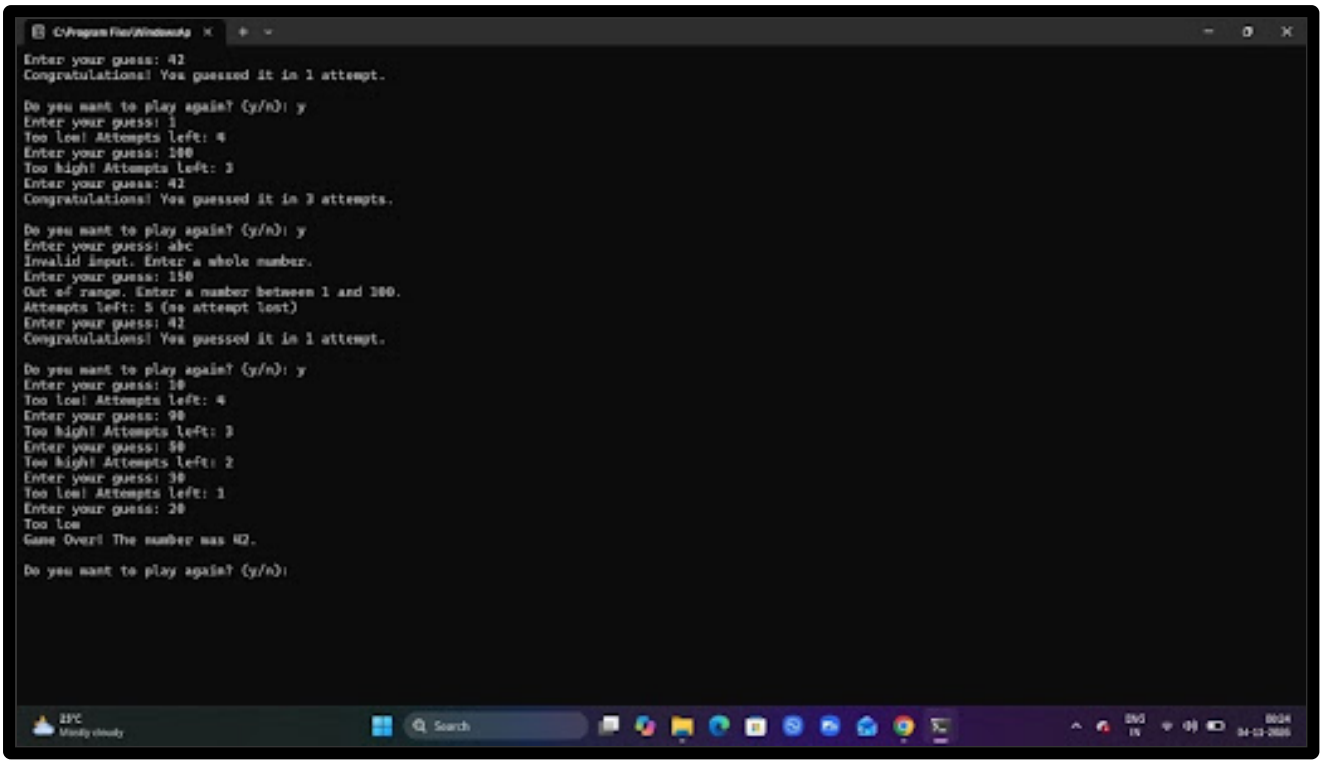
    # Step 4: Game Over Condition
    if attempts_left == 0 and guess != secret_number:
        print(f"Game Over! The number was {secret_number}.")

def main():
    while True:
        play_game()
        play_again = input("\nDo you want to play again? (y/n): ").strip().lower()
        if play_again != 'y':
            print("Thanks for playing! Goodbye 👋")
            break

if __name__ == "__main__":
    main()
```

PROGRAM EXECUTION OUTPUT & TERMINAL SCREENSHOT

Below is the high-clarity console execution output demonstrating all required test cases and game flows:



EXECUTION TEST CASES (4 SCENARIOS)

Test Case	Input Scenario	Expected & Observed Output
Test Case 1: Normal Input	Secret number: 42 Guess: 42	`Congratulations! You guessed it in 1 attempt.`
Test Case 2: Boundary / Limits	Secret number: 42 Guesses: 1, 100	`Too low! Attempts left: 4` `Too high! Attempts left: 3`
Test Case 3: Invalid Input	Secret number: 42 Guesses: abc, 150	`Invalid input. Enter a whole number.` `Out of range... Attempts left: 5 (no attempt lost)`
Test Case 4: Exhausted Attempts	Secret number: 42 Guesses: 10, 90, 50, 30, 20	`Too low | Too high | Too high | Too low | Too low` `Game Over! The number was 42.`